Laboratorio 12

Problema 1

Adrian Lopez – 231361

5 de noviembre de 2026

1. Reducción-β de NOT

En cálculo lambda los booleanos se representan como funciones de dos argumentos que escogen uno de ellos (booleanos de Church):

$$\mathsf{TRUE} \equiv \lambda x.\lambda y.\, x \qquad\qquad \mathsf{FALSE} \equiv \lambda x.\lambda y.\, y$$

Como un booleano escoge entre dos opciones, para negarlo basta con pasarle las opciones al revés:

$$\mathsf{NOT} \equiv \lambda p.\, p\ \mathsf{FALSE}\ \mathsf{TRUE}$$

La regla de reducción-β es $(\lambda x.M)\, N \to_\beta M[x := N]$, es decir, se reemplaza el parámetro por el argumento dentro del cuerpo.

NOT TRUE

$$\begin{aligned}
\mathsf{NOT\ TRUE} &\equiv (\lambda p.\, p\ \mathsf{FALSE}\ \mathsf{TRUE})\ (\lambda x.\lambda y.\, x) \\
&\to_\beta (\lambda x.\lambda y.\, x)\ \mathsf{FALSE}\ \mathsf{TRUE} && [p := \mathsf{TRUE}] \\
&\to_\beta (\lambda y.\, \mathsf{FALSE})\ \mathsf{TRUE} && [x := \mathsf{FALSE}] \\
&\to_\beta \mathsf{FALSE} && [y := \mathsf{TRUE}] \\
&\equiv \lambda x.\lambda y.\, y
\end{aligned}$$

Escribiendo todos los términos (se renombran las variables de FALSE y TRUE a a, b para que no se confundan con las de NOT TRUE):

$$\begin{aligned}
&(\lambda p.\, p\ (\lambda a.\lambda b.\, b)\ (\lambda a.\lambda b.\, a))\ (\lambda x.\lambda y.\, x) \\
&\to_\beta (\lambda x.\lambda y.\, x)\ (\lambda a.\lambda b.\, b)\ (\lambda a.\lambda b.\, a) \\
&\to_\beta (\lambda y.\lambda a.\lambda b.\, b)\ (\lambda a.\lambda b.\, a) \\
&\to_\beta \lambda a.\lambda b.\, b\ \equiv_\alpha\ \lambda x.\lambda y.\, y = \mathsf{FALSE}
\end{aligned}$$

NOT FALSE

$$\begin{aligned}
\mathsf{NOT\ FALSE} &\equiv (\lambda p.\, p\ \mathsf{FALSE}\ \mathsf{TRUE})\ (\lambda x.\lambda y.\, y) \\
&\to_\beta (\lambda x.\lambda y.\, y)\ \mathsf{FALSE}\ \mathsf{TRUE} && [p := \mathsf{FALSE}] \\
&\to_\beta (\lambda y.\, y)\ \mathsf{TRUE} && [x := \mathsf{FALSE}] \\
&\to_\beta \mathsf{TRUE} && [y := \mathsf{TRUE}] \\
&\equiv \lambda x.\lambda y.\, x
\end{aligned}$$

Otra forma de definir NOT es $\mathsf{NOT} \equiv \lambda p.\lambda a.\lambda b.\, p\ b\ a$, que intercambia el orden de los argumentos:

$$\begin{aligned}
\mathsf{NOT\ TRUE} &\equiv (\lambda p.\lambda a.\lambda b.\, p\ b\ a)\ (\lambda x.\lambda y.\, x) \\
&\to_\beta \lambda a.\lambda b.\, (\lambda x.\lambda y.\, x)\ b\ a \\
&\to_\beta \lambda a.\lambda b.\, (\lambda y.\, b)\ a \\
&\to_\beta \lambda a.\lambda b.\, b = \mathsf{FALSE}
\end{aligned}$$

2. Recursión y ciclos

En cálculo lambda no hay nombres globales ni variables que se puedan modificar. Una función no se puede llamar a sí misma por su nombre y no existen `for` ni `while`. Las dos cosas se tienen que construir solo con funciones.

2.1. Recursión

La recursión se logra con un combinador de punto fijo, como el combinador Y:

$$Y \equiv \lambda f.\, (\lambda x.\, f\ (x\ x))\ (\lambda x.\, f\ (x\ x))$$

Lo importante de Y es que $Y\ g =_\beta g\ (Y\ g)$:

$$\begin{aligned}
Y\ g &\equiv (\lambda f.\, (\lambda x.\, f\ (x\ x))\ (\lambda x.\, f\ (x\ x)))\ g \\
&\to_\beta (\lambda x.\, g\ (x\ x))\ (\lambda x.\, g\ (x\ x)) && [f := g] \\
&\to_\beta g\ ((\lambda x.\, g\ (x\ x))\ (\lambda x.\, g\ (x\ x))) && [x := \lambda x.\, g\ (x\ x)] \\
&=_\beta g\ (Y\ g)
\end{aligned}$$

El último paso se cumple porque la expresión entre paréntesis es justo a la que reduce $Y\ g$ en el primer paso. Entonces g recibe como primer argumento "una copia de sí misma" y la puede usar para hacer la llamada recursiva. Por ejemplo, el factorial con numerales de Church:

$$\begin{aligned}
F &\equiv \lambda f.\lambda n.\ \mathsf{IF}\ (\mathsf{ISZERO}\ n)\ 1\ (\mathsf{MULT}\ n\ (f\ (\mathsf{PRED}\ n))) \\
\mathsf{FACT} &\equiv Y\ F
\end{aligned}$$

Desarrollo de $\mathsf{FACT}\ 2$ ($\to_\beta^*$ significa varios pasos de reducción):

$$
\begin{aligned}
\mathsf{FACT}\ 2 &\equiv Y\ F\ 2 \\
&=_\beta F\ (Y\ F)\ 2 \\
&\to_\beta^* \mathsf{IF}\ (\mathsf{ISZERO}\ 2)\ 1\ (\mathsf{MULT}\ 2\ ((Y\ F)\ (\mathsf{PRED}\ 2))) \qquad [f := Y\ F,\ n := 2] \\
&\to_\beta^* \mathsf{MULT}\ 2\ (Y\ F\ 1) \\
&=_\beta \mathsf{MULT}\ 2\ (F\ (Y\ F)\ 1) \\
&\to_\beta^* \mathsf{MULT}\ 2\ (\mathsf{MULT}\ 1\ (Y\ F\ 0)) \\
&=_\beta \mathsf{MULT}\ 2\ (\mathsf{MULT}\ 1\ (F\ (Y\ F)\ 0)) \\
&\to_\beta^* \mathsf{MULT}\ 2\ (\mathsf{MULT}\ 1\ 1) \\
&\to_\beta^* 2
\end{aligned}
$$

Esto funciona con orden normal (se reduce primero lo de afuera). En un lenguaje que evalúa los argumentos antes de llamar a la función, como JavaScript, Y se queda en un ciclo infinito, por eso se usa el combinador $Z \equiv \lambda f.\,(\lambda x.\,f\ (\lambda v.\,x\ x\ v))\ (\lambda x.\,f\ (\lambda v.\,x\ x\ v))$:

```
const Z = f => (x => f(v => x(x)(v)))(x => f(v => x(x)(v)));
const fact = Z(rec => n => n === 0 ? 1 : n * rec(n - 1));
fact(5); // 120
```

Aquí `fact` nunca se llama a sí misma por su nombre, la recursión la da Z.

2.2. Ciclos

Un ciclo se escribe como una función recursiva. Las variables que el ciclo iría cambiando pasan a ser parámetros, la condición de salida es el caso base y cada iteración es una llamada recursiva con los valores nuevos (recursión de cola). Por ejemplo, para sumar $1 + 2 + \cdots + n$:

```
// imperativo
let acc = 0;
for (let i = n; i > 0; i--) acc += i;

// funcional
const suma = (i, acc = 0) => i === 0 ? acc : suma(i - 1, acc + i);
suma(4); // suma(3,4) -> suma(2,7) -> suma(1,9) -> suma(0,10) -> 10
```

En cálculo lambda se puede definir un `while` general usando Y:

$$\mathsf{WHILE} \equiv Y\ (\lambda w.\lambda c.\lambda s.\lambda e.\ \mathsf{IF}\ (c\ e)\ (w\ c\ s\ (s\ e))\ e)$$

donde c es la condición, s es el paso y e es el estado. Mientras $c\ e$ sea TRUE se vuelve a llamar con el estado nuevo $s\ e$, y cuando es FALSE se devuelve e.

Además, los numerales de Church ya funcionan como ciclos. El número n es la función que aplica f exactamente n veces:

$$0 \equiv \lambda f.\lambda x.\,x \qquad 1 \equiv \lambda f.\lambda x.\,f\ x \qquad 2 \equiv \lambda f.\lambda x.\,f\ (f\ x) \qquad n \equiv \lambda f.\lambda x.\,f^n\ x$$

Así que $n\ f\ x$ es lo mismo que repetir f n veces empezando en x, como un `for` de n iteraciones. Por ejemplo $\mathsf{ADD} \equiv \lambda m.\lambda n.\,m\ \mathsf{SUCC}\ n$ suma aplicando SUCC m veces a n.

```
const tres = f => x => f(f(f(x)));
tres(x => x + 2)(0); // 6
```

En la práctica los lenguajes funcionales tienen funciones como `map`, `filter` y `reduce` que reemplazan la mayoría de ciclos.

3. ¿Cuándo usar programación funcional y cuándo no?

Conviene usarla cuando el problema es transformar datos: se tiene una entrada que pasa por varios pasos hasta dar una salida. Como las funciones son puras (misma entrada, misma salida) y no se modifica estado compartido, el código es más fácil de probar y se puede paralelizar sin preocuparse por condiciones de carrera.

Ejemplo: calcular el total vendido de una categoría a partir de una lista de ventas. Cada paso es una función pura y el trabajo se podría repartir entre varios procesadores (es la idea de MapReduce).

```
const total = ventas
  .filter(v => v.categoria === 'hogar')
  .map(v => v.precio * v.cantidad)
  .reduce((acc, x) => acc + x, 0);
```

No conviene cuando el problema es principalmente estado que cambia todo el tiempo, o cuando el rendimiento y la memoria son críticos, porque crear copias nuevas en lugar de modificar en el mismo lugar cuesta tiempo y memoria. Tampoco conviene usar recursión muy profunda en lenguajes que no optimizan la recursión de cola, como JavaScript o Java, porque se desborda la pila (por ejemplo `suma(100000)` de la sección anterior da `RangeError: Maximum call stack size exceeded`).

Ejemplo: el ciclo principal de un videojuego que mueve miles de partículas 60 veces por segundo. Hacerlo de forma funcional pura obliga a crear un arreglo nuevo con miles de objetos en cada cuadro, lo cual le da mucho trabajo al recolector de basura y causa pausas. Es más simple y eficiente modificar las partículas directamente:

```
for (const p of particulas) {
  p.x += p.vx * dt;
  p.y += p.vy * dt;
}
```